OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Авторизация и аутентификация в бэкенде

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в TG
OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Auth types
- Federated Auth
- Встраивание в логику
- Тестирование
- Рефлексия

Цели вебинара

После занятия вы:

1. Узнаете особенности разработки авторизационной подсистемы приложения
2. Сможете работать с авторизацией на базе OpenID
3. Сможете встраивать авторизацию в бизнес-логику приложения, тестировать и запускать

Смысл

Зачем вам это уметь

1. Практически все web-приложения имеют систему аутентификации/авторизации
2. Практически все развитые современные системы поддерживают OpenID
3. Есть много особенностей интеграции системы авторизации в логику приложения

О защите информации

Защита информации

Особенности:

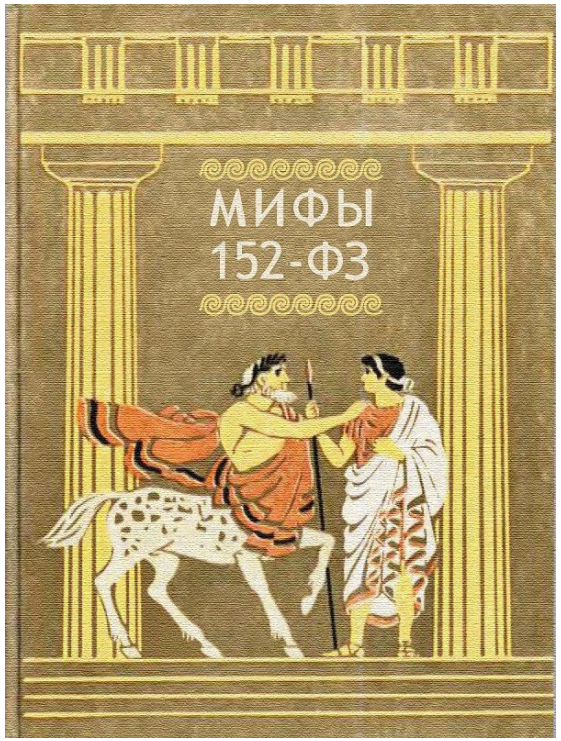
1. Бюрократия
2. Множество ограничений
3. Подвержены все из-за ПДН
4. Обязательна сертификация ФСТЭК для:
   - ГИС
   - ЗОКИИ
   - ИСПДн
   - АСУТП
5. Необходимо учитывать при разработке архитектуры и системы бэкенда

Защита информации

Обход 152-фз:

1. Никак – штрафы Роскомнадзора, судебные иски от пользователей
2. Выполнять буквально – очень дорого
3. Аутсорсить часть инфраструктуры
4. Пользоваться обезличенной информацией – ПДН хранить в других организациях, псевдонимы вместо ФИО и т.д.
5. Пользоваться общедоступной информацией, пользователи соглашаются на раскрытие ПДН(*)

* это не освобождает от множества других ограничений



Auth Types

Аутентификация, авторизация

Идентификация – процесс распознавания пользователя по его идентификатору

Аутентификация – проверка подлинности

Авторизация – проверка наличия прав

Auth Types

Наиболее известные типы аутентификации:

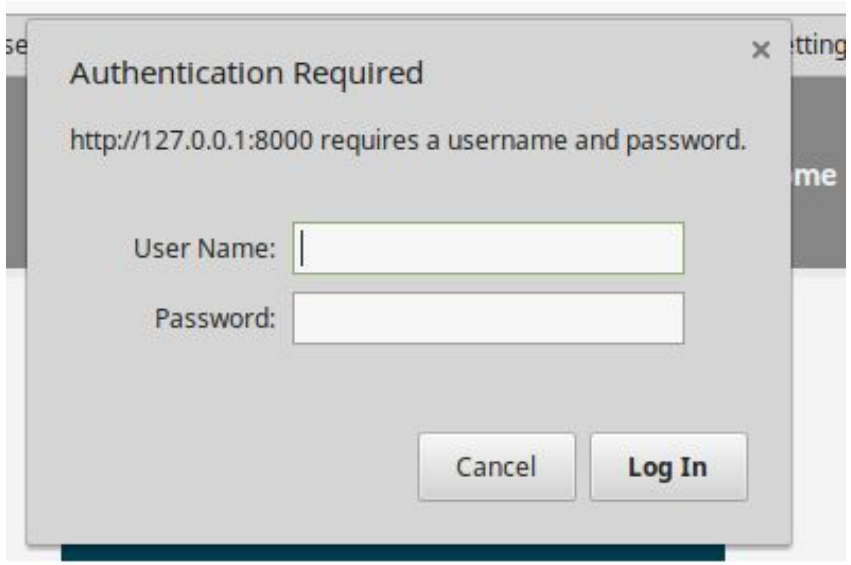
- Basic
- Digest
- User Session
- Сертификат (Kerberos, Radius)
- OpenID, OIDC (OpenID Connect)
- OAuth (1 и 2)
- ~~LDAP~~!!!!!!!

Многофакторая аутентификация

Basic Auth

Общие сведения:

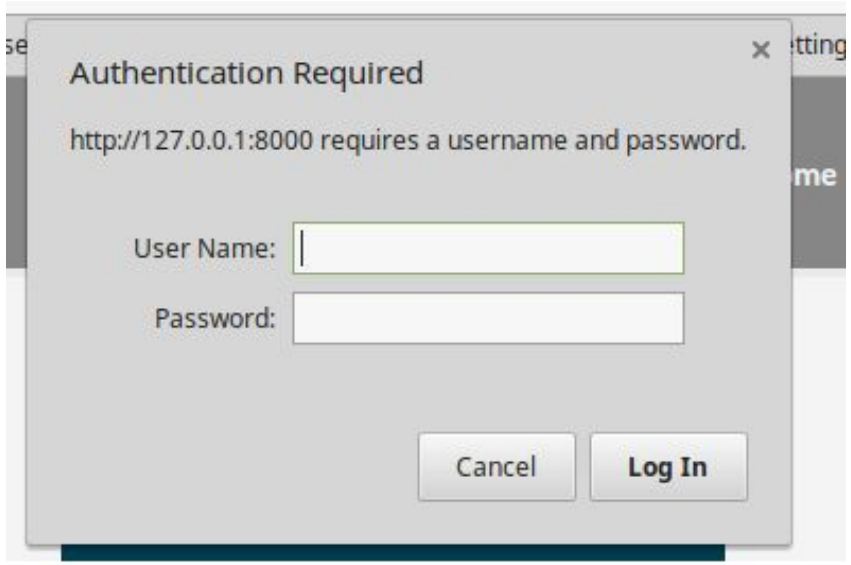
- Стандартизован
- Выполняется на уровне браузера
- Реализован с помощью заголовка: Authorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==
- Base64 user:password
- Рекомендуется использовать SSL



Digest Auth

Общие сведения:

- Стандартизован
- Выполняется на уровне браузера
- Реализован с помощью MD5 хэшей



$$\mathrm{HA1} = \mathrm{MD5}\left(\mathrm{A1}\right) = \mathrm{MD5}\left(\mathrm{username} : \mathrm{realm} : \mathrm{password}\right)$$

$$\mathrm{HA2} = \mathrm{MD5}\left(\mathrm{A2}\right) = \mathrm{MD5}\left(\mathrm{method} : \mathrm{digestURI}\right)$$

$$\mathrm{response} = \mathrm{MD5}\left(\mathrm{HA1} : \mathrm{nonce} : \mathrm{HA2}\right)$$

Недостатки парольной аутентификации

Недостатки:

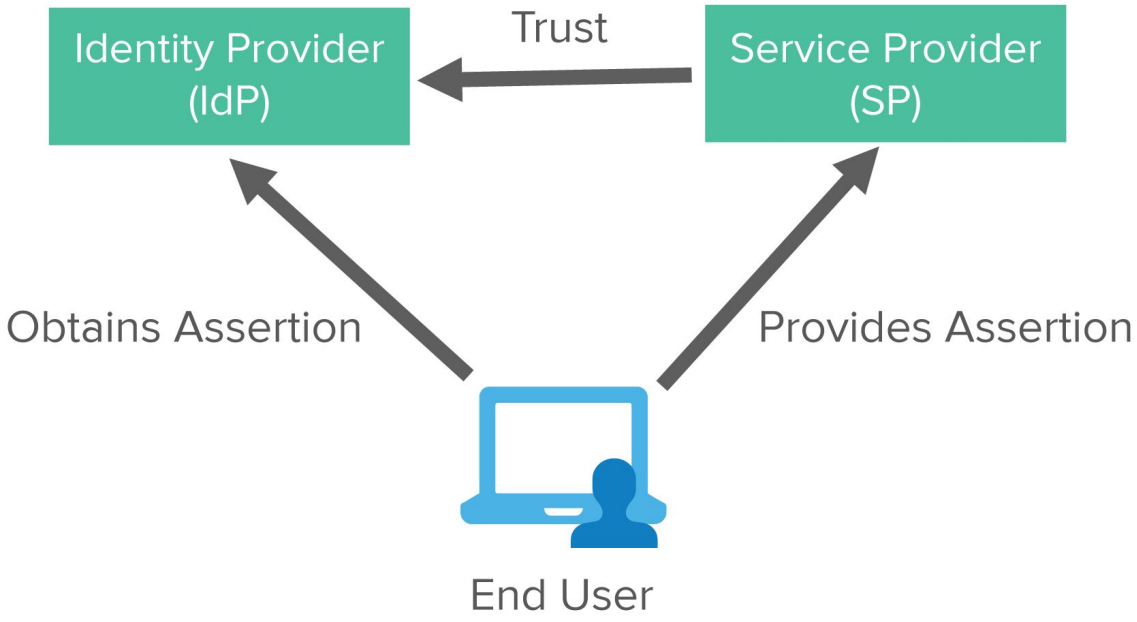
- Трудности запоминания
- Низкое качество паролей
- Дорогое управление учетными данными (персональные данные, восстановление пароля, многофакторная аутентификация и пр.)
- Особые требования к ПДН (персональным данным)
- Проблемы с защищенностью в микросервисной среде

Федеративная аутентификация

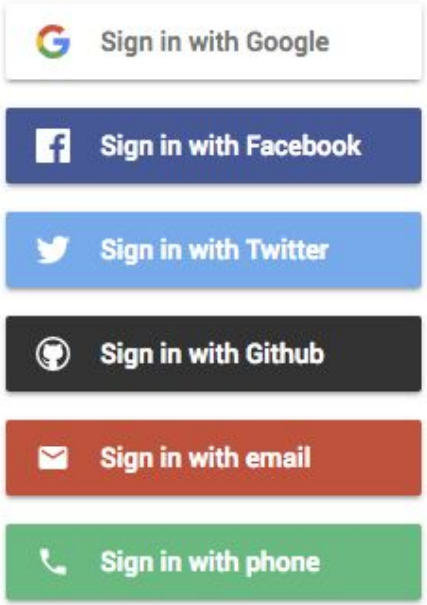
Федеративная аутентификация

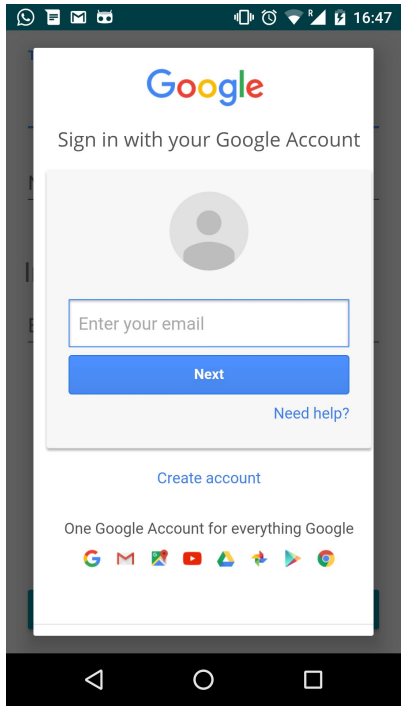
Типы:

- OAuth2 (авторизация)
- OIDC (аутентификация)
- JWT – не протокол!



Федеративная аутентификация

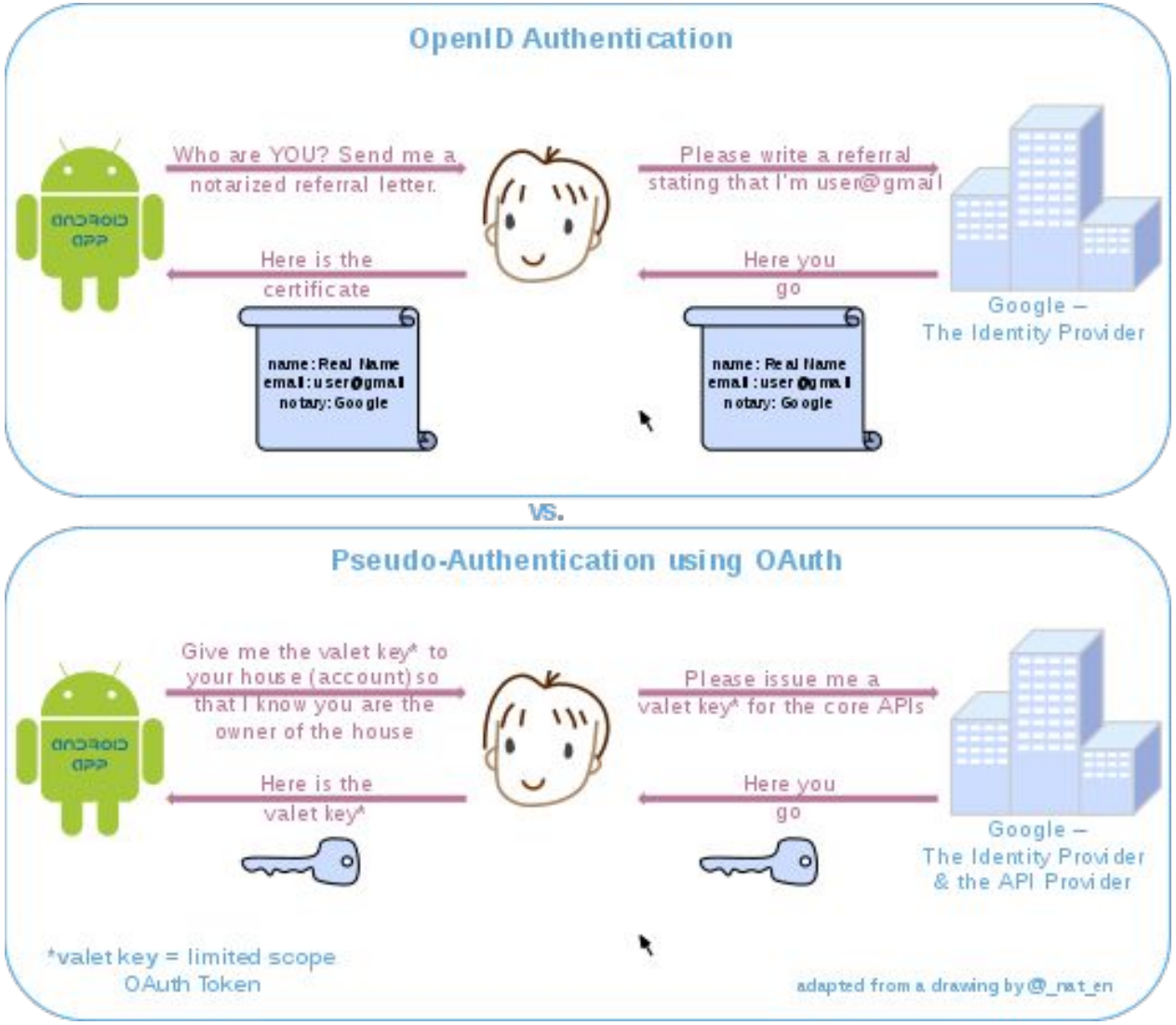




OpenID Connect

Алгоритм:

- User пытается получить доступ к Resource Server (RS)
- RS перенаправляет User на Authentication Server (AS)
- Пользователь вводит login/pass, получает XRI подписанный AS документ
- User идет с XRI по обратному адресу на RS
- Получает доступ к ресурсам



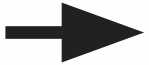
JWT

Чтобы каждый не придумывал свой формат токенов, возник стандарт JWT.

JWT-токен – это просто закодированный в base64 с данными + заголовок с метаинформацией и подпись.

JWT = B64(Header).B64(Payload).SIGN

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c



HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Header will include encryption algorithm

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

Payload (also referred as **claims**) will include data that we want to transfer

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret
) ☐ secret base64 encoded
```

Signature is created by signing encoded header and payload with signature using header algorithm

https://jwt.io/

JWT Payload

Данные в Payload должны приходить не в произвольном формате, и есть набор специальных или зарегистрированных атрибутов (claims) и возможность использовать свои.

Например:

- **iss** – кто выпустил токен
- **sub** – о ком данный токен
- **exp** – дата протухания

и другие

https://www.iana.org/assignments/jwt/jwt.xhtml

JWT библиотеки

В подавляющем большинстве случаев самостоятельно писать библиотеки для кодирования и декодирования JWT токенов не придется (и не стоит)

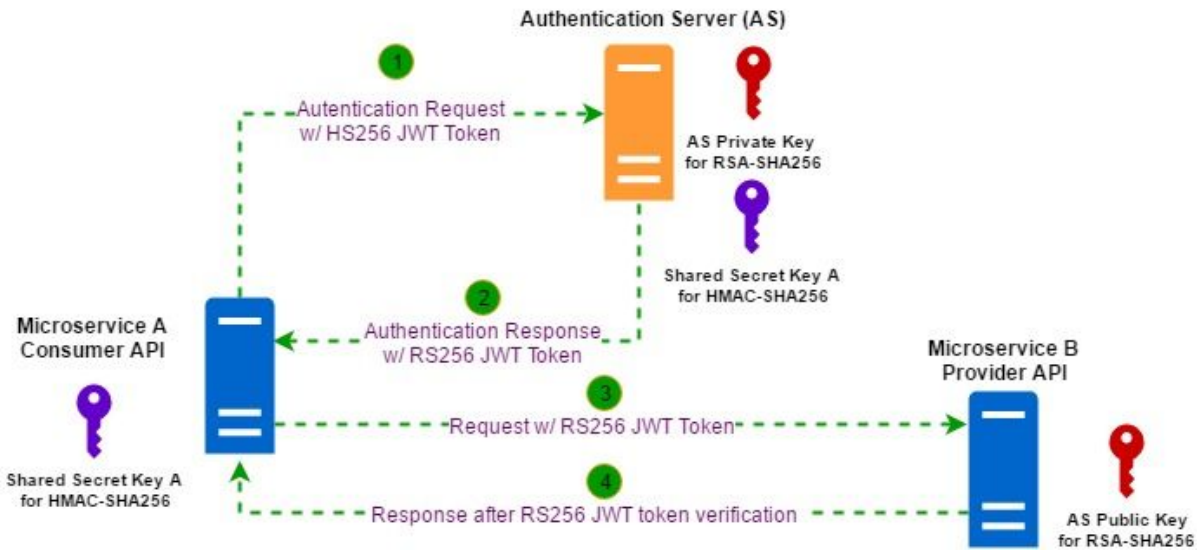
Здесь - https://jwt.io/#libraries-io – можно посмотреть набор официальных библиотек.

Kotlin Multiplatform НЕТ

Микросервисная архитектура

Алгоритм:

- Сервис A хочет сходить в сервис B
- Сервис A идет в AS, аутентифицируется и получает access-токен и refresh-токен
- Сервис A добавляет access-токен в заголовок запроса к сервису B
- Сервис B проверяет токен (токен подписан) и отвечает
- Когда access-токен устаревает (несколько минут или несколько десятков минут), сервис A получает новый access токен по имеющемуся refresh-токену



Интеграция в приложение

C4 - схема контейнеров

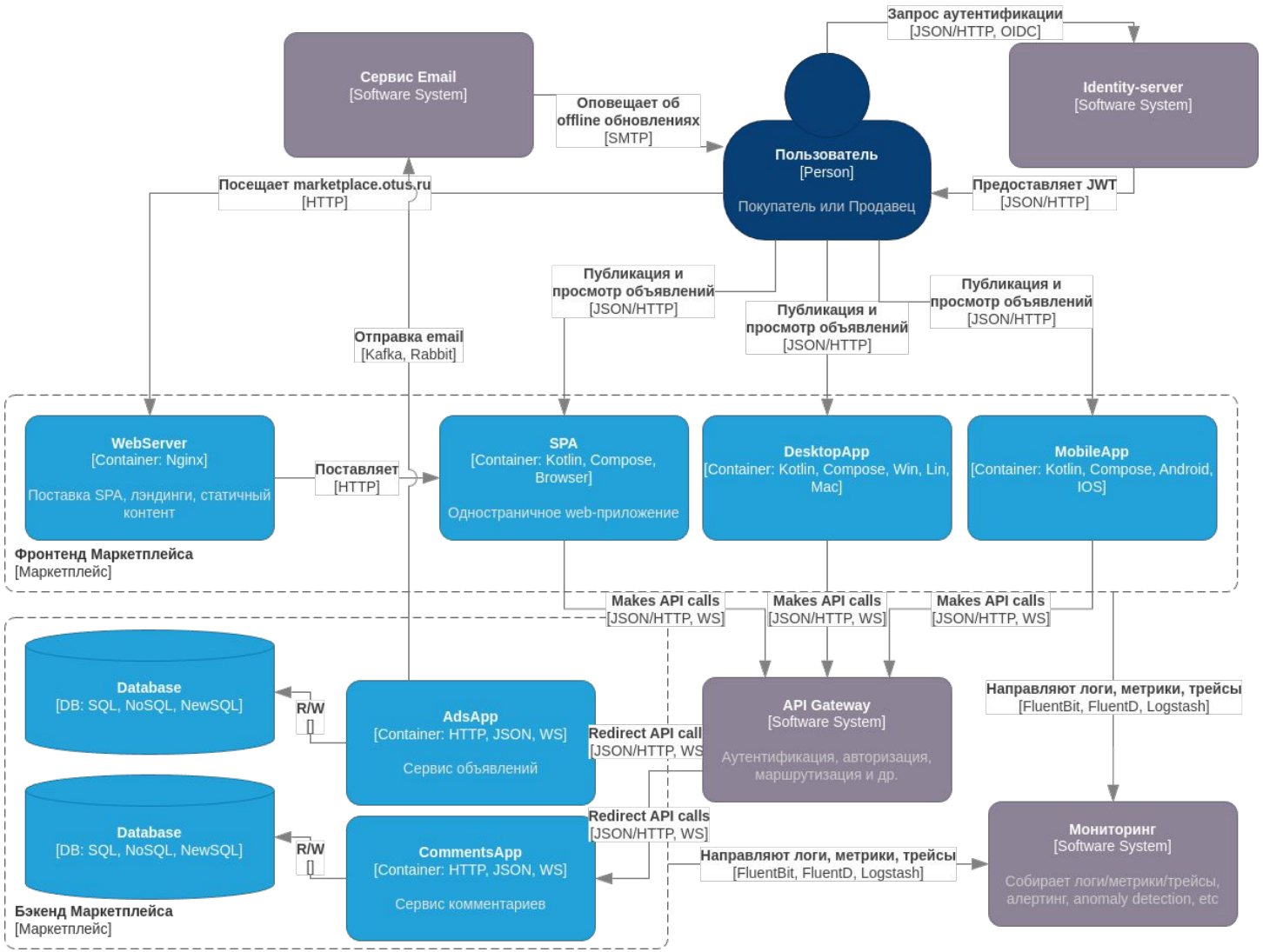
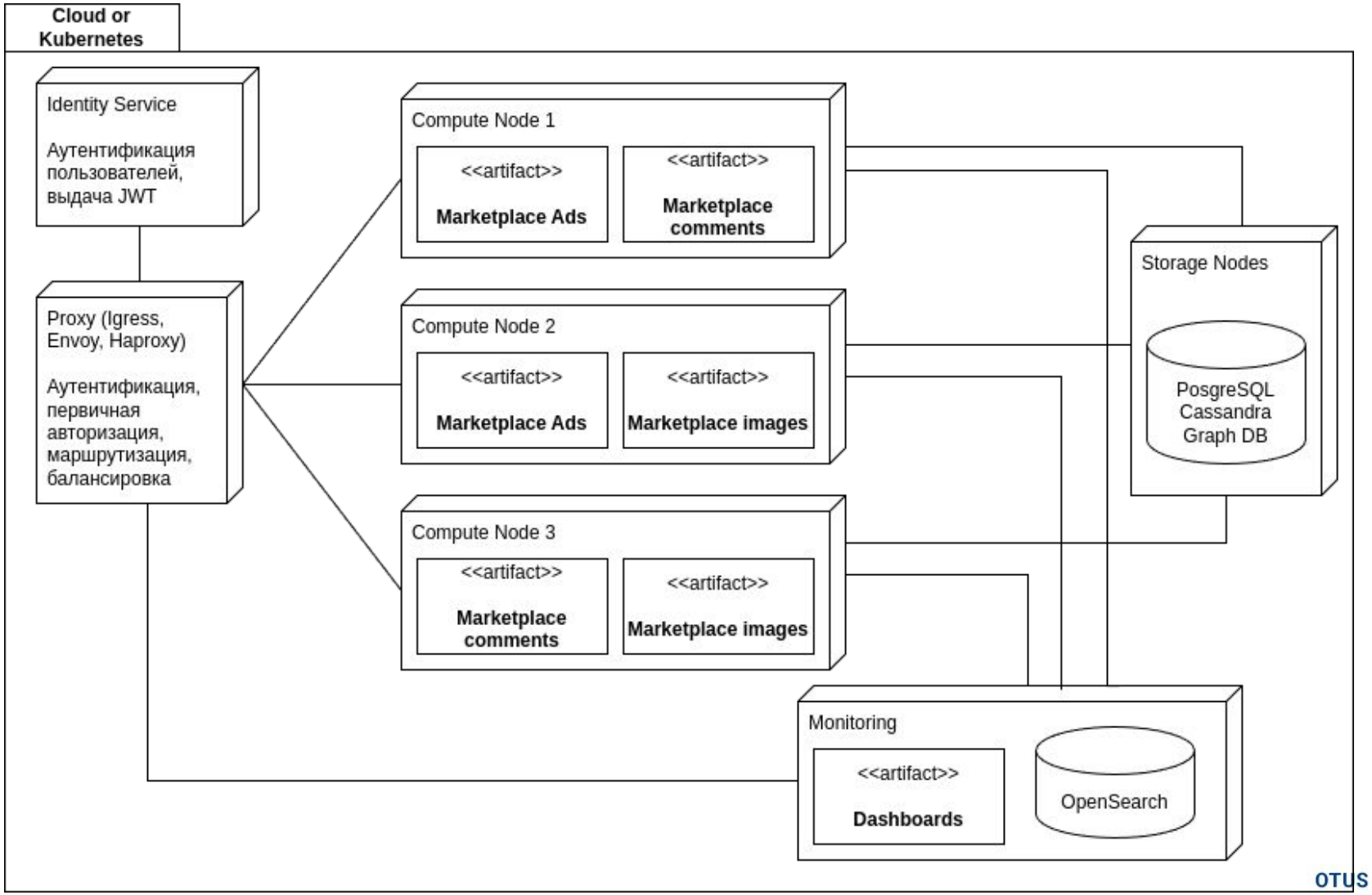
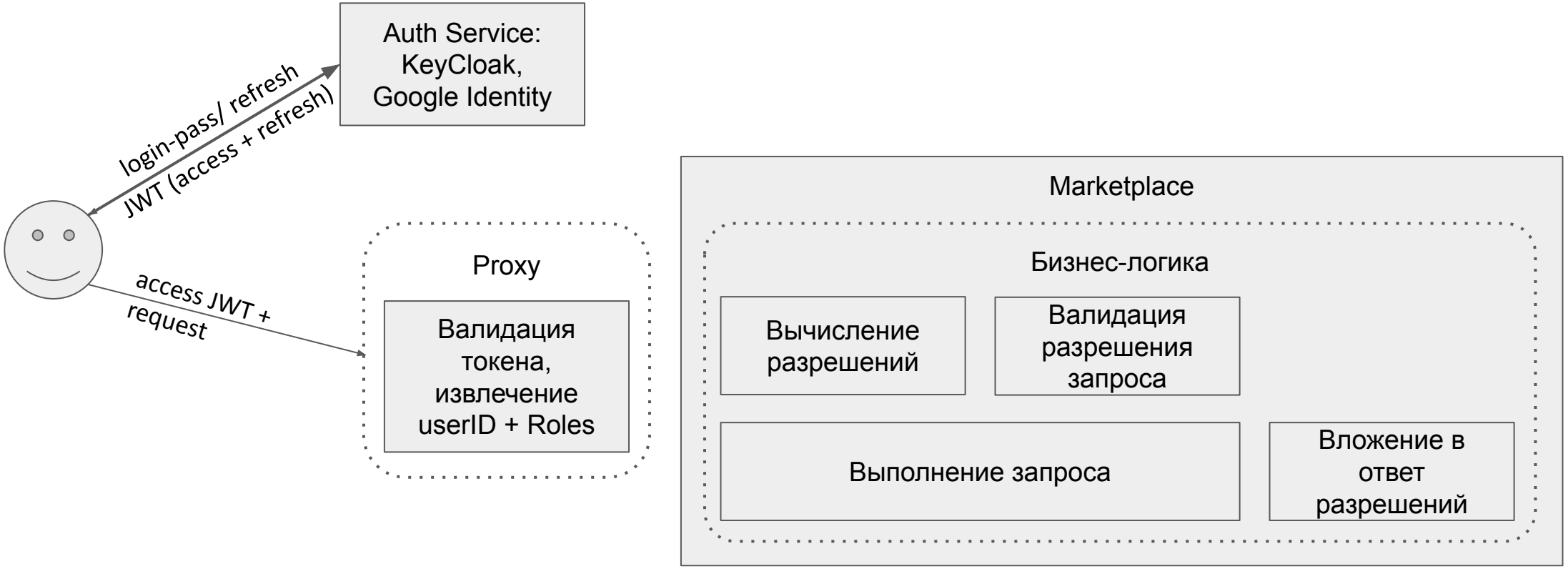


Схема развертывания



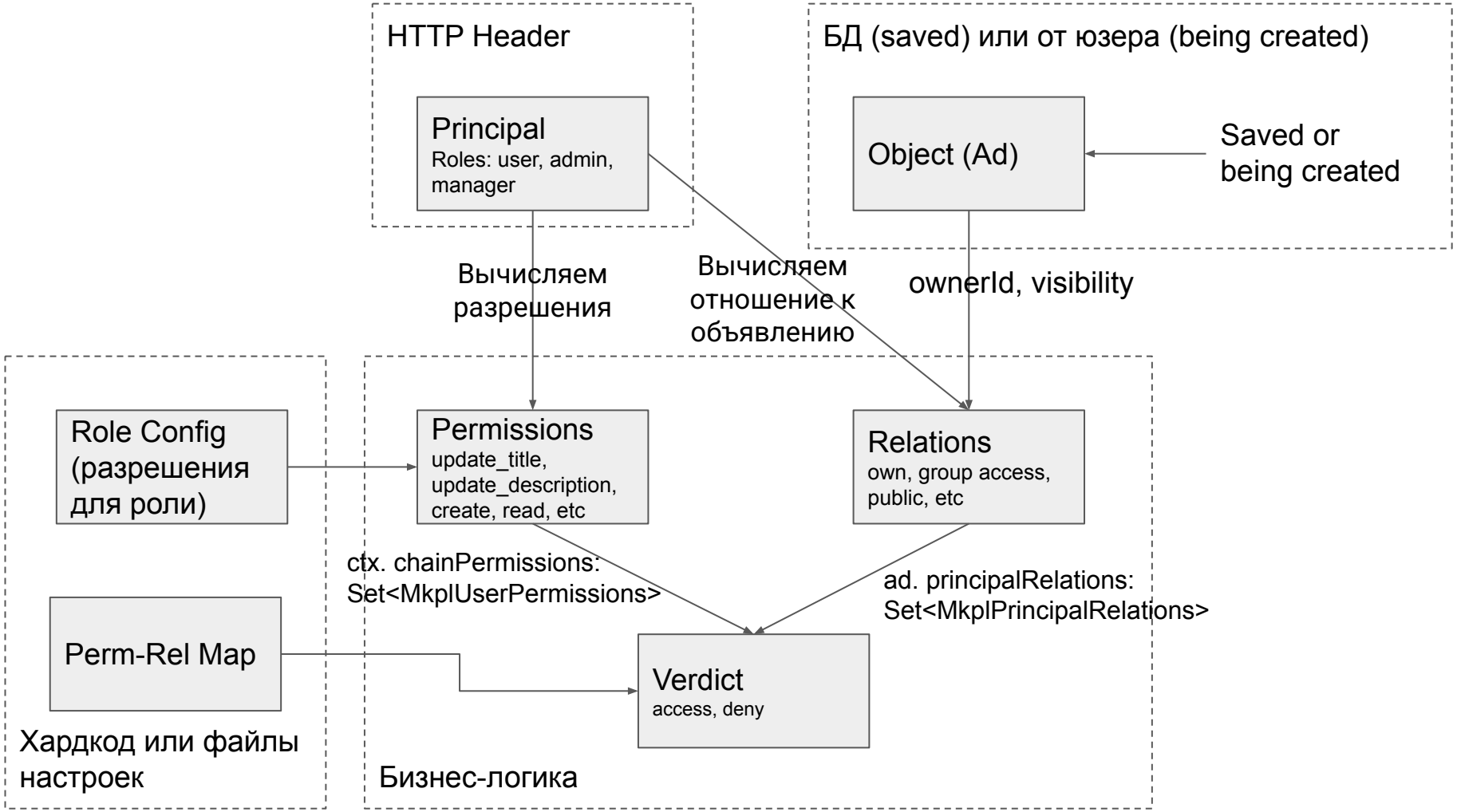
Архитектура аутентификации



Алгоритм:

- Пользователь получает access+refresh JWT
- Отправляет запрос в сервис, сопровождая access JWT
- Обновление access JWT на сервере аутентификации с refresh JWT

Алгоритм авторизации



LIVE

Рефлексия

Тезисы

Подведем итоги

1. Типы аутентификации: Basic, Digest, сессионная, OAuth, OpenID, LDAP
2. Федеративная аутентификация работает через посредника – Authentication Service
3. OAuth отдает RS “ключи”, а OpenID – подписанный документ с user-info
4. Интеграция: федеративная аутентификация, маппинг в контроллере, правила – в чейне
5. Обычные тесты для бизнес логики + интеграционный тест

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Цели вебинара

Проверка достижения целей:

1. особенности разработки авторизационной подсистемы приложения
2. Сможете работать с авторизацией на базе OpenID
3. Сможете встраивать авторизацию в бизнес-логику приложения, тестировать и запускать

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/202505-otuskotlin-marketplace
4. https://dzone.com/articles/how-to-use-kotlin-to-create-a-secure-ktor-applicat
5. https://github.com/myndocs/kotlin-oauth2-server
6. https://en.wikipedia.org/wiki/OpenID
7. https://jwt.io/
8. https://github.com/Slenkis/ktor-full-jwt
9. https://medium.com/slickteam/ktor-and-keycloak-authentication-with-openid-ecd415d7a62e
10. Envoy: https://www.envoyproxy.io/docs/envoy/latest/
11. Keycloak: https://www.keycloak.org/documentation

Следующий вебинар

24.10.2025

Модуль 8: Развитие бизнес-логики проекта
Введение в машинное обучение

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары